

? AWS FinOps Project

Project Title - AWS FinOps Cost Management & Optimization

SECTION 1 - Project Overview

Business Scenario

A small business runs a web application on AWS and wants better visibility, control, and optimization of its cloud spending.

Objective

Build a small AWS environment and apply FinOps practices to monitor, analyze, control, and optimize cloud costs.

Key Goals

- Secure AWS resources
- Create a basic workload
- Monitor resource usage
- Track AWS costs
- Set cost controls
- Apply tagging
- Identify optimization opportunities
- Document FinOps recommendations

AWS Services Planned

Category	AWS Service	Purpose
Compute	EC2	Run our demo application
Storage	S3	Store project files/data
Identity & Security	IAM	Control access
Monitoring	CloudWatch	Monitor resources
Cost Analysis	Cost Explorer	Analyze AWS spending
Cost Control	AWS Budgets	Set spending alerts

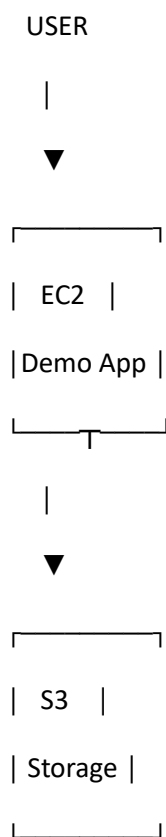
Category	AWS Service	Purpose
Cost Allocation	Tags	Track resources/costs

SECTION 2- Architecture

Architecture – Conceptual Design

This architecture represents the conceptual relationship between the AWS resources used in the FinOps project. The project focuses on AWS cost management and monitoring rather than deploying a production web application.

Then write this simple flow:



IAM —————▶ Access & Security

CloudWatch —————▶ Monitoring

Cost Explorer —▶ Cost Analysis

AWS Budgets —► Cost Alerts

Tags —► Cost Allocation

SECTION 3 - AWS Environment & Implementation

3.1 EC2 — Compute Layer

EC2 is the compute layer of the project. It provides the virtual server on which our application/workload will run.

Configuration:

- AMI: Amazon Linux 2023
- Instance type: t3.micro
- Storage: 8 GiB encrypted gp3 EBS volume
- Security Group: Project security group with SSH restricted to My IP
- Key Pair: finops-project-key

FinOps consideration: A small t3.micro instance was selected as a cost-conscious configuration for the lightweight project workload. Actual utilization will be monitored before making any further rightsizing decision.

3.2 S3 — Object Storage Layer

Amazon S3 is used as the object storage layer of the project. It provides separate, scalable storage for project files and documentation rather than storing them directly on the EC2 instance.

Configuration:

- Bucket: finops-project-storage-103942587895-ap-south-1-an
- Region: Mumbai (ap-south-1)
- Bucket type: General purpose
- Object Ownership: Bucket owner enforced / ACLs disabled
- Block Public Access: Enabled
- Versioning: Enabled
- Encryption: SSE-S3
- Tags: Project, Environment, Application
- Object uploaded: Project Word document

FinOps/Security consideration: The bucket is kept private, public access is blocked, versioning helps recover previous versions of objects, and server-side encryption protects stored objects.

3.3 - IAM & Security

IAM Security Configuration: Existing IAM user used for AWS access. MFA is enabled using a passkey for additional account security.

IAM Configuration

- Existing IAM user used for project activities
- MFA enabled (Passkey)
- AdministratorAccess currently assigned
- No additional IAM user created

3.4 Cost Baseline

- ☐ AWS account with Free Tier eligibility/usage
- ☐ Budget monitoring enabled
- ☐ Cost monitoring through AWS Cost Explorer
- ☐ AWS Budgets used for cost alerts
- ☐ Project costs will be monitored throughout implementation

AWS Budget & Cost Alert

Budget: \$1 monthly

Alert threshold: \$0.01 actual cost

Email recipient: Configured

Automated action: Not configured

Purpose: To monitor actual AWS spending and receive an early notification when costs exceed the defined threshold.

3.5 Cost Allocation Tags

Project resources were tagged using Project, Environment, and Application tags to support resource identification, organization, and cost allocation. These tags were activated as cost allocation tags in AWS Billing and Cost Management so they can be used for cost analysis and reporting.

Activated tags:

- Project → AWS-FinOps
- Environment → Learning
- Application → FinOps-Demo

3.6 EC2 Instance Deployment

Instance name: finops-project-ec2

AMI: Amazon Linux 2023, Kernel 6.1

Instance type: t3.micro

Storage: 8 GiB gp3 encrypted root volume

Security Group: finops-project-sg

SSH access: Restricted to My IP

Key pair: finops-project-key
Public IP: Enabled
Credit specification: Standard
Status: Running; 3/3 status checks passed

3.7 CloudWatch Monitoring

CloudWatch was configured to monitor the CPU utilization of the finops-project-ec2 instance. A CloudWatch alarm named finops-project-ec2-high-cpu was created with an average CPU utilization threshold of 70% over a 5-minute period. The alarm is intended to identify unusually high EC2 CPU utilization and support proactive monitoring.

Current alarm state: OK. After the EC2 instance was started, CloudWatch began receiving CPU utilization metrics and the alarm transitioned from Insufficient data to OK.

3.8 EC2 Cost Optimization

1. Right-sizing

Continue monitoring CPU utilization before making a rightsizing decision. Current utilization is very low, but the observation period is too short to confidently recommend changing the instance type.

2. Stop when not required

Stop the EC2 instance when the environment is not being used to avoid unnecessary compute charges.

3. Keep storage minimal

The current 8 GiB gp3 EBS volume is small and appropriate for the project workload. No immediate storage rightsizing is required.

SECTION 4 Analysis & FinOps Findings-

4.1 Our EBS finding

The EC2 instance uses a single 8 GiB encrypted EBS root volume. The volume is currently attached and configured to delete upon EC2 termination. No immediate storage optimization opportunity was identified. The delete-on-termination configuration helps prevent unintended accumulation of orphaned storage.

4.2 S3 finding

The project S3 bucket currently contains one 22.4 KB object stored in S3 Standard. The storage footprint is minimal and the object is actively relevant to the project. No immediate storage-class optimization or lifecycle action is justified. Lifecycle policies could be considered if the bucket grows or accumulates older, infrequently accessed objects.

4.3 EC2 Analysis

- Instance type: t3.micro
- vCPU: 2
- Memory: 1 GiB
- Current CPU utilization observed: approximately 0.107%–0.213%
- CloudWatch alarm threshold: 70%
- Current utilization is very low
- The instance was recently started, so there isn't enough historical data for a reliable rightsizing decision.
- Recommendation: continue monitoring CPU utilization over a longer period before changing the instance type.

4.4 CloudWatch Analysis

- CloudWatch monitors EC2 CPU utilization
- Metric: CPUUtilization
- Alarm: finops-project-ec2-high-cpu
- Threshold: CPUUtilization $\geq$ 70%
- Evaluation: 1 datapoint within 5 minutes
- Current alarm state: **OK**
- Purpose: identify unusually high CPU utilization and support proactive monitoring
- Current finding: no high-CPU condition detected

4.5 Cost Explorer Analysis

- Cost Explorer was used to analyze AWS spending
- Analysis period: last 7 days
- Granularity: Daily
- Group by: Service
- No additional service/region/instance filters for the baseline view
- Current billable cost observed: **\$0**
- Project tag AWS-FinOps was also checked
- Project cost observed: **\$0**
- Finding: no current billable cost was observed during the analysis period

4.6 Cost Allocation Analysis

- Cost allocation tags activated:
 - Project
 - Environment
 - Application
- Tag values:
 - Project → AWS-FinOps
 - Environment → Learning
 - Application → FinOps-Demo
- These tags help identify and allocate costs to resources/projects
- Finding: the project has a basic cost allocation structure in place

4.7 AWS Budget Analysis

- Budget name: My Zero-Spend Budget
- Budget type: Monthly
- Monthly budget: **\$1**
- Actual cost: **\$0**
- Alert threshold: **Actual cost > \$0.01**
- Budget status: **OK / Healthy**
- Automated actions: None
- Purpose: provide an early warning against unexpected AWS spending
- Finding: current spending is within the configured budget

4.8 Overall FinOps Findings

- Cost visibility has been established through Cost Explorer
- Cost allocation has been established through tagging
- Spending control has been established through AWS Budgets
- Resource utilization monitoring has been established through CloudWatch
- EC2 currently shows very low CPU utilization
- EBS and S3 storage usage is minimal
- No immediate optimization change is justified based on the current data
- More historical usage data is required before making a firm rightsizing decision

SECTION 5- FinOps Recommendations

- 🔍 Continue monitoring EC2 utilization before rightsizing.
 - 🔍 Stop EC2 when it is not required to avoid unnecessary compute charges.
 - 🔍 Keep EBS storage minimal and remove unused volumes/resources.
 - 🔍 Review S3 storage periodically and introduce lifecycle policies if the amount of older data grows.
 - 🔍 Maintain consistent resource tagging.
 - 🔍 Continue monitoring Cost Explorer and AWS Budget alerts.
- 🔍 Review AWS resources periodically for unused or underutilized resources.

SECTION 6- Project Outcome

- AWS environment created
- EC2 workload configured
- S3 storage configured
- IAM security and MFA implemented
- CloudWatch monitoring implemented
- Cost Explorer used for cost analysis
- AWS Budget configured
- Cost allocation tags implemented
- EBS and S3 analyzed
- EC2 utilization analyzed
- FinOps optimization opportunities and recommendations identified

SECTION 7 - Conclusion

The project demonstrates how FinOps principles can be applied to an AWS environment through cost visibility, monitoring, budgeting, tagging, resource analysis, and optimization recommendations.

